
Governance Framework — CloudServe Support Automation

Author: Madhav Mehta

What must be true before this system speaks to a paying customer without a person reading it first.

The test applied throughout: pick a way the system could harm a customer, and name what in the design would stop it. Where the answer describes a property of the model rather than a control that was built, it is recorded as a hope, not a mitigation.

1. Decision logging

Every automated decision is written to `storage/decisions.db` (or the run's output directory during evaluation). One record per ticket on every path — answered, escalated, blocked, or failed internally.

The record

Implemented in `src/db.py`, carrying the Governance Framework's minimum schema plus three additions.

Field	Purpose
<code>decision_id</code> , <code>timestamp</code> , <code>ticket_id</code>	Identity
<code>stage</code>	Which part of the pipeline decided
<code>input_summary</code>	First 200 characters of what arrived
<code>model_name</code> , <code>model_version</code>	What produced the prediction
<code>prediction_value</code> , <code>prediction_confidence</code>	Intent and its confidence
<code>readiness_score</code>	The number the threshold was applied to
<code>alternatives_json</code>	What else was considered, with confidences
<code>sources_used_json</code>	Every retrieved passage with its score
<code>threshold_applied</code>	The threshold in force at the time
<code>action_taken</code>	<code>auto_respond</code> , <code>escalate</code> or <code>block</code>
<code>reason</code>	Why, in language a support manager can read
<code>guardrail_results_json</code>	What was checked and what fired
<code>prompt_version</code>	Which prompt version produced it
<code>requirement_ids_json</code>	Which requirements the behaviour serves
<code>channel</code> , <code>customer_tier</code> , <code>language_fluency</code>	Added so the fairness audit is a query against the log, not a separate exercise

Field	Purpose
latency_ms , degraded	Operational context

prompt_version and requirement_ids exist because the question after any incident is "was this behaviour intended?", and only those two fields answer it.

The segment fields matter more than they look. A fairness audit that requires assembling a separate dataset is an audit that gets run once, for the report, and never again. Because tier, channel and fluency are on every record, the audit is a SQL query and runs on every evaluation automatically.

Coverage

reconcile() checks three things, not one: total decisions equals tickets processed, distinct tickets equals tickets processed, and no ticket appears twice. Counting only distinct tickets would hide a double-logged ticket, which is the more insidious fault because it inflates confidence in coverage.

On the recorded run: 80 decisions, 80 distinct tickets, 0 duplicates. PASS.

TestA8DecisionLog::test_reconciliation_detects_a_gap deliberately under-reports the ticket count and asserts MISMATCH, because a check that cannot fail is not a check.

2. Risk register

ID	Risk	Likelihood	Impact	Mitigation in the design	Owner
R-01	The system answers confidently and incorrectly	Medium	Severe	Calibrated readiness score (error 4.24 pts, verified); answers grounded only in retrieved passages; every citation checked against what retrieval returned and the response blocked if any does not resolve; hard escalation when no passage clears the relevance threshold	Head of Support
R-02	Private data appears in an outbound response	Low	Severe	Six pattern families checked on every response; blocks and escalates rather than redacting , so the underlying fault is not masked; zero occurrences on the recorded run; probe 1 verifies the block fires	Head of Support
R-03	A customer's input is treated as an instruction	Medium	Severe	Two independent controls: ticket text fenced in <code><ticket></code> tags and passed in a separate message role from instructions; plus a pattern-based input validator that blocks and records the input. Probes 1 and 2 verify	Engineering
R-04	Some customer groups receive worse answers	Occurring	Severe	Measured on every run by tier, channel and fluency. Currently failing at 20 points against a 5-point requirement. See §3	Head of Support
R-05	The documentation the system relies on goes stale	Medium	Moderate	Retrieval restricted to Ines Varga's 29 reviewed articles; never learns from agent snippet files (Daniel: answers "correct two years ago and not right since"); doc_id logged on every answer, so a bad article is traceable to every reply that used it	Technical Writer

ID	Risk	Likelihood	Impact	Mitigation in the design	Owner
R-06	The model provider becomes unavailable	High	Low, as designed	Local grounded generation is a supported mode, not an emergency path — the entire recorded evaluation ran through it. Timeout, error status and malformed response all degrade to it; four tests induce each	Engineering
R-07	Latency degrades under load	Low	Moderate	p95 of 5.42 ms against a 3 s target, over 500× headroom. Retrieval is in-memory; no network call in the default path	Engineering
R-08	Costs rise unexpectedly with volume	Low	Low	Runs with no provider configured at all. Provider use is optional and capped at 400 output tokens	Head of Support
R-09	A ticket that must never be automated is automated	Medium	Severe	Policy rules on intent applied <i>before</i> any confidence is consulted, plus a veto when a never-automate intent scores within 20% of the top prediction. Zero unsafe automations at every threshold in the sweep	Head of Support
R-10	The system closes tickets without solving them	Medium	Moderate	Not mitigated in the design. Repeat contacts run at 21.6% historically and cannot be measured without live deployment. Recorded as the first metric to instrument after launch	Head of Support
R-11	Stakeholders trust a metric that is a proxy	Medium	Moderate	Citation accuracy and hallucination rate are labelled as proxies wherever they appear; CSAT is not reported at all rather than being invented	Author

R-10 is listed deliberately without a mitigation. It is a real risk with no control behind it, and recording it honestly is more useful than describing a hope.

3. Fairness audit

Method. Every ticket in the validation run is segmented by customer tier, language fluency and channel, and the resolution rate compared across segments. Segments below five tickets are excluded as too small to support a conclusion. Computed automatically by `evaluation/metrics.py` on every run; no separate exercise, no manual sampling.

Sample: 80 validation tickets, the full set.

Segment	Tickets	Resolution rate	Variation from best
By customer tier			
Business	30	86.67%	—
Enterprise	8	75.0%	11.67 pts
Standard	42	66.67%	20.0 pts
By language fluency			
Non-fluent	19	84.21%	—
Fluent	61	72.13%	12.08 pts
By channel			

Segment	Tickets	Resolution rate	Variation from best
Email	31	83.87%	—
Forum	11	72.73%	11.14 pts
Docs comment	16	68.75%	15.12 pts
Chat	22	68.18%	15.69 pts

Finding: the requirement is not met

Worst variation: 20.0 points, against a requirement of under 5. This condition fails. The Project Brief is explicit that governance conditions are not targets to approach but conditions that hold or do not, and that a system failing one is not fit to deploy whatever its other numbers say. On this measure the system is not deployable as it stands.

What is not the cause. The router does not read customer tier except for one enterprise-urgency rule, and only eight enterprise tickets are in the sample. There is no rule that treats standard customers differently. The gap is emergent, arising from what standard-tier customers write about rather than from how they are handled.

What I did not do about it. The available fix — tier-aware thresholds — would close the measured gap by explicitly applying a different standard according to what a customer pays. Ravi Menon's warning was precisely that a widening gap between plans "will come up at renewal", and engineering the metric while making the underlying treatment more unequal would be worse than reporting it. The better fix is to understand why standard-tier tickets are harder, and that was not done.

Two notes on reading this table. The enterprise segment has eight tickets and should carry very little weight. And the fluency result runs *opposite* to the direction the Governance Framework predicts — retrieval systems usually perform worse on non-fluent phrasing, and here non-fluent tickets do better. That matches the historic record in the development corpus, where non-fluent customers also fared slightly better, so it is consistent rather than an artefact. It does not mean the concern is unfounded; it means it does not appear at this sample size.

4. Guardrails

Each runs on every response, in the evaluation harness exactly as in the API. There is no flag that disables them.

Guardrail	What it checks	What happens when it fires
Kill switch	Whether automated responding is suspended	Block, escalate
Instruction integrity	Whether ticket text tries to alter the system's instructions	Block, escalate, record the input for review
Tone and scope	Commitments about refunds, credits, quotas, dates, guarantees	Block. These are not the system's to make
Private data	National insurance and card numbers, API secrets, bearer tokens, private keys	Block and escalate. Never redact and send

Guardrail	What it checks	What happens when it fires
Citation grounding	Every cited document resolves to a passage retrieval returned	Block, with the unsupported citation named
Confidence floor	The threshold was actually applied	Block. A missing confidence score is not a high one

Checks are not short-circuited on the first failure: the log is more useful recording everything wrong with a response than only the first thing noticed.

Evidence they are live

The validation corpus contains no adversarial tickets, so a run over it fires nothing — which proves nothing either way. After each run the harness pushes **ten engineered probes through the same pipeline instance**, so a guardrail disabled for the run would be caught immediately. Probe decisions go to a separate audit log, since synthetic records in the run's own log would be indistinguishable from the duplicate-logging fault reconciliation exists to catch.

Recorded run: 10 of 10 probes behaved as required, including a control case confirming that an acceptable response is *not* blocked — a guardrail that blocks everything is as useless as one that blocks nothing.

5. Incident response

Written to be followed at two in the morning by somebody who did not build it.

Step	What to do	Who	How long
1. Detect	Alert, customer complaint, or agent report. Confirm by querying the decision log: <code>SELECT * FROM decisions WHERE ticket_id = '<id>'</code> — it returns exactly what the system saw, predicted, retrieved, and why it acted	Duty agent	5 min
2. Contain	<code>curl -X POST localhost:8000/killswitch -H 'Content-Type: application/json' -d '{"active": true, "operator": "<your name>", "reason": "<what you saw>"}</code> . Takes effect on the next response. No restart, no deployment. Tickets in flight finish their current step then escalate rather than send	Duty agent — no authorisation needed	2 min
3. Assess	Scope: <code>SELECT COUNT(*) FROM decisions WHERE action_taken='auto_respond' AND timestamp > '<when it started>'</code> . Common cause: the same <code>prompt_version</code> , the same <code>doc_id</code> in <code>sources_used_json</code> , or the same intent. If one article is implicated, list every reply that used it	Engineering	30 min
4. Notify	Head of Support always. Affected customers where a wrong answer was sent. Legal where private data or a commitment was involved	Head of Support	1 h
5. Remediate	Wrong article → Technical Writer corrects it; replies that cited it are identifiable from the log. Prompt fault → fix, bump <code>prompt_version</code> , re-run the harness. Guardrail gap → add the pattern, add a probe, confirm the probe fails before the fix and passes after	Engineering	Same day

Step	What to do	Who	How long
6. Review	Within five working days: what the log did and did not tell you. Any question the log could not answer is a schema gap and a change to <code>src/db.py</code>	Head of Support	5 days

Kill switch — the details

Question	Answer
What is the mechanism?	<code>POST /killswitch {"active": true}</code> , or <code>GuardrailEngine.kill_switch = True</code> in process
Who is authorised?	Any duty agent. Deliberately unrestricted: requiring authorisation to stop a system that is already causing harm adds delay at the worst moment. The operation is logged with the operator's name, so it is accountable after the fact rather than gated before it
How long to take effect?	The next response. No restart, no deployment
What happens to tickets in flight?	They complete the current step, then the validator blocks the response and they escalate. Nothing is dropped, and every one is logged
How is it tested?	<code>TestA7Guardrails::test_kill_switch_blocks_everything</code> , and <code>demo.py</code> scenario 6 engages it, shows a block, releases it and shows service resume

6. The declaration

Statement	Position
This system must never...	send a customer an answer it cannot trace to a specific reviewed support article; make any commitment about money, quotas or dates; answer a security incident, compliance request, feature request or ticket it cannot understand; or send a response containing anyone's private data
The mechanism that enforces that is...	for grounding, citation checks that block on any unresolvable reference plus an escalation when nothing clears the relevance threshold; for commitments, pattern checks that block before sending, independently of the prompt instruction; for the four intents, policy rules applied before confidence is consulted, plus a veto when one is a close runner-up; for private data, blocking rather than redaction. Each is a control, and each has a probe that fires it
The most likely way it could still cause harm is...	by giving a confident, well-cited answer drawn from an article that has become wrong since it was last reviewed. Every control here checks that an answer is <i>grounded</i> ; none checks that the ground is still true. <code>last_reviewed_days_ago</code> exists in the corpus and the system does not use it. The second most likely is the 20-point resolution gap quietly becoming a commercial problem before anyone connects it to a renewal conversation
We would not deploy this without first...	closing the cross-group variation to under 5 points, or obtaining an explicit written decision from CloudServe that a 20-point gap is acceptable and why; agreeing what a wrong automated answer costs, since the threshold is not robust to that assumption above about 20×; instrumenting repeat contacts, which is the only measure that would reveal tickets being closed without being solved; and validating the classifier on real traffic rather than a synthetic corpus whose separability flatters it